TP5: Programmation dynamique et mémoïsation

CPES ≪ Sciences des données, arts et cultures ≫

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

07 janvier 2025

Dans ce TP, nous allons résoudre quelques problèmes en utilisant la programmation dynamique, ainsi que la mémoïsation.

Suite de Fibonacci

On considère la suite de Fibonacci, qui pour rappel est définie par la formule de récursion suivante : $F_0 = 0$, $F_1 = 1$ et $F_n = F_{n-1} + F_{n-2}$, $\forall n \geqslant 2$.

Question 1 Justifier pourquoi l'utilisation de la programmation dynamique est adaptée à ce problème.

Question 2 Écrire, en utilisant la programmation dynamique, une fonction Python `suite_fibonacci_dynamique`, prenant en argument la valeur `n`, indice auquel on souhaite calculer la suite. Cette fonction doit retourner la valeur F_n.

Question 3 Quelles sont les complexités temporelles et spatiales de cette fonction en *notation asymptotique* $\Theta()$? Justifier (sans calculs détaillés).

Rendu de monnaie (nombre minimal de pièces)

On considère le problème du nombre minimum de pièces à rendre, vu en cours, que l'on veut résoudre avec la **mémoïsation**. Il est définit comme suit : on veut rendre une somme d'argent $S \in \mathbb{N}$ en utilisant le système de pièces $P = (v_1, v_2, \ldots, v_n)$ triées par ordre croissant, tel que $|P| = n$. La solution à ce problème est une suite $R^* = (x_1, x_2, \ldots, x_n)$, où, $\forall i \in [n]$, $v_i \in \mathbb{N}$, tel que :

$$\sum_{i=1}^{n} v_i \times x_i = S$$

$$R^* \in \underset{(x_1,\ldots,x_n)}{\operatorname{argmin}} \left(\sum_{i=1}^{n} x_i \right)$$

Dans cet exercice, on s'intéressera uniquement à la caractérisation du minimum, et non à la façon dont celui-ci est obtenu. Une extension naturelle et intéressante à ce problème est de déterminer *l'argument* associé au minimum, à savoir les valeurs des variables pour lesquelles le minimum est atteint. Concrètement ici, il s'agirait de la combinaison de pièces à rendre, qui en pratique est la donnée qui est la plus intéressante.

Afin de résoudre ce problème, on se concentre sur la formule de récurrence suivante :

$$N(S,k) = \min\left(N(S,k-1), N(S-v_k,k)+1\right)$$

, où $N(S,k)$ est le nombre mimimal de pièces à rendre sur la somme S en utilisant les k premières pièces du système P. Ainsi, déterminer le nombre minimal de pièces à rendre pour la somme S_1, c'est déterminer $N\left(S_1,n\right)$, étant donné que P est de taille n.

Question 4 Expliquer, en quelques mots, le sens de cette formule de récurrence.

Question 5 En gardant en tête que N représente un nombre de pièces minimal à rendre, déterminer les cas de base (d'arrêt) associés à cette formule de récurrence.

Question 6 Écrire, en utilisant la mémoïsation, une fonction `nombre_minimal_pieces_memoisation` prenant en argument :

- le système de pièces `P` (une liste Python $(v_1, v_2, \ldots, v_n)$, rangée par ordre croissant)
- la somme à rendre `S` (un entier positif ou nul)
- la position `k` de la dernière pièce de notre système P dans sa liste. *Exemple : si* `P = [1, 2, 5]`*, alors* $k = 2$.
- une structure de données permettant la mémoïsation, ici sous la forme d'un dictionnaire. Les clés seront des tuples Python `(S, k)`, où s est la somme associée à la sous-structure et k le nombre de pièces dans le système associé à la sous-structure. A chaque clé sera associée une valeur entière représentant le nombre minimal de pièces associé à cette sous-structure. *Exemple : si notre structure contient la clé* `(13, 3)`*, et que celle-ci est associée à la valeur* `2`*, alors cela signifie que pour la somme* $S = 13$*, on rendra 2 unités de la quatrième pièce du système* P *(puisque* P *est une liste indexée à partir de 0).*

Point important : comme votre fonction fera des appels récursifs, il vous faut exécuter `sys.setrecursionlimit(100000)`, en ayant bien pensé, avant d'écrire cela, à avoir importé la bibliothèque `sys` avec `import sys`. Cette ligne permet de modifier la limite du nombre d'appels récursifs à des sous-procédures que Python fixe, qui est par défaut assez basse.

Pour cette question, vous devez utiliser la formule de récurrence donné dans l'énoncé. Tester votre code avec le système de « pièces » Européen $P_{\text{Euro}} = (1,2,5,10,20,50,100,200)$, et la somme $S = 1301$. Même chose avec le système de pièces $P_{\text{petites_coupures}} = (1,2,5)$ et la somme $S = 113$.

Problème du sac à dos

Dans cet exercice, on revient sur le problème du sac à dos, déjà abordé lors du TP sur les algorithmes gloutons. Voici un rappel de sa formalisation.

On considère un sac à dos de capacité C et $n \in \mathbb{N}$ objets différents, dont on note $p_0, \ldots, p_{n-1}$ les poids et $v_0, \ldots, v_{n-1}$ les valeurs. On considère que ces poids et valeurs sont *des entiers strictement positifs*, et que le poids de chaque objet ne dépasse pas la capacité totale. Le but du problème est de retourner un sous-ensemble $X \subseteq [\![0, n-1]\!]$ tel que :

— $\sum_{k \in X} p_k \leqslant C$

— $\sum_{k \in X} v_k$ est maximale.

L'objectif de cet exercice est de déterminer **la valeur maximale** qu'il est possible d'obtenir avec une capacité C et les objets indiqués.

Question 7 Selon le même principe que dans l'exercice précédent, *mais en gardant en tête que l'objectif et les éléments d'entrée sont différents*, proposer une formule de récurrence et le ou les cas de base. La récurrence portera sur le calcul de $V(C, k)$, à savoir la valeur obtenue étant donnée une capacité C et les k premiers objets (pris par ordre croissant de poids).

Question 8 Utiliser la programmation dynamique afin d'écrire une fonction Python `sac_a_dos_dynamique` prenant en entrée une liste Python des poids `P`, une liste Python des valeurs `V` et la capacité `C`.

Question 9 Quelle est la complexité temporelle de cette fonction ? Justifier.